DESARROLLO DE SOFTWARE · C#

Fundamentos de C# para desarrollo de software

Una base clara para modelar datos, construir expresiones y controlar el flujo de un programa.

10 CONCEPTOS · SINTAXIS · CONSOLA



Made with GAMMA

Tipos de datos: elegir la representación correcta

Enteros

`int` para cantidades sin decimales.

Decimales

`double` o `decimal` para valores con precisión.

Texto

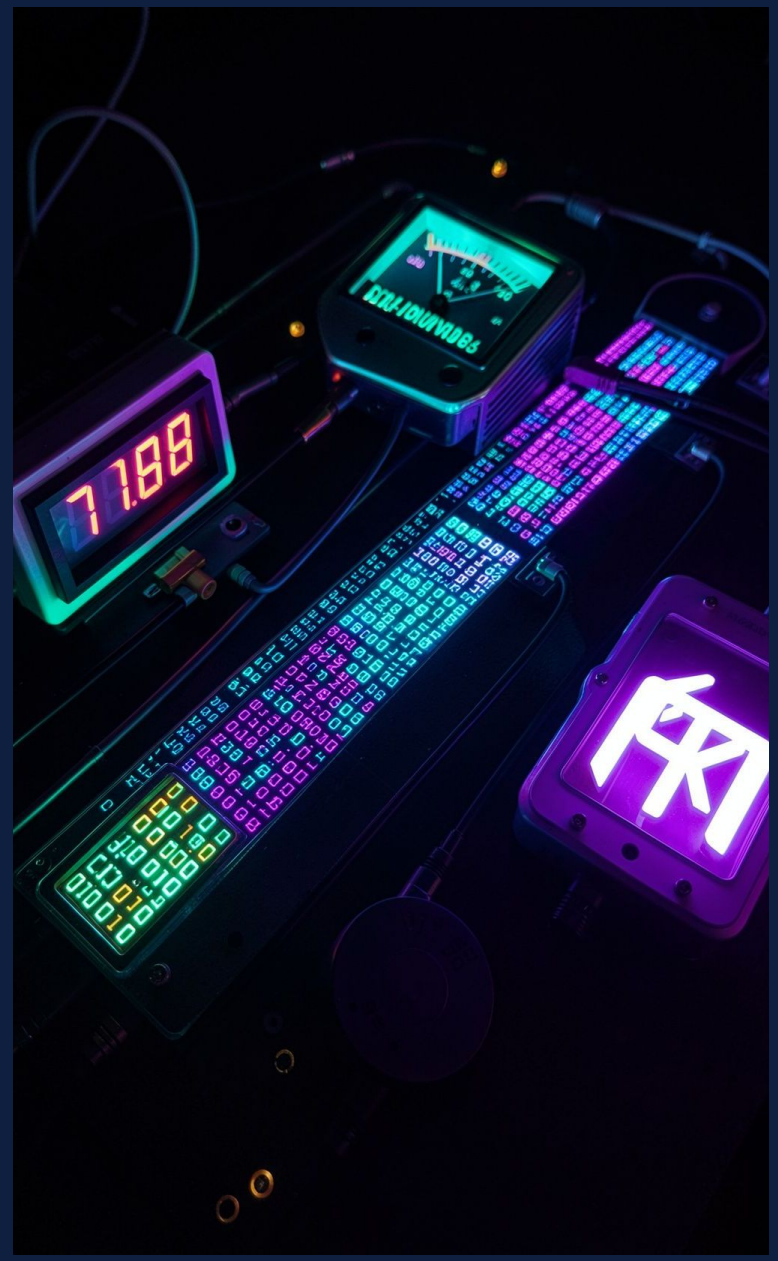
`string` para cadenas.

Lógicos

`bool` para `true` o `false`.

Carácter

`char` para un símbolo individual.



La representación correcta hace

Made with GAMMA

Variables: memoria con nombre y propósito

Acción	Sintaxis o práctica
Declarar	`int edad;`
Inicializar	`int edad = 20;`
Inferir tipo cuando conviene	`var total = 125.50;`
Nombres y alcance	Usar nombres claros y mantener un alcance controlado

Made with GAMMA

Operadores: construir expresiones

Los operadores transforman valores y permiten expresar una regla de cálculo.

Operadores

- **Suma:** +
- **Resta:** -
- **Multiplicación:** *
- **División:** /
- **Módulo:** % para obtener el residuo

PRECEDENCIA E INTENCIÓN

Respetar la precedencia y usar paréntesis para expresar intención.

```
resultado = a + b * c;
```

```
resultado = (a + b) * c;
```

Made with GAMMA

04 · SALIDA

La consola responde.

Imprimir es hacer visible el estado del programa para la persona y para quien lo prueba.

Impresión en pantalla: hablar con la consola

WRITE LINE

```
Console.WriteLine("Listo");
```

Escribe y cambia de línea.

WRITE

```
Console.Write("Total: ");
```

Escribe sin salto de línea.

INTERPOLACIÓN

```
$"Total: {total}"
```

Inserta valores dentro del texto.

Separa presentación, datos y lógica para facilitar las pruebas.

Made with GAMMA

Entrada por teclado: convertir texto en datos

La consola entrega texto. El programa debe convertirlo y decidir qué hacer si no es válido.

01

Solicitar

```
Console.
Write("Ed
ad: ");
```

02

Leer

```
Console.R
eadLine()
```

03

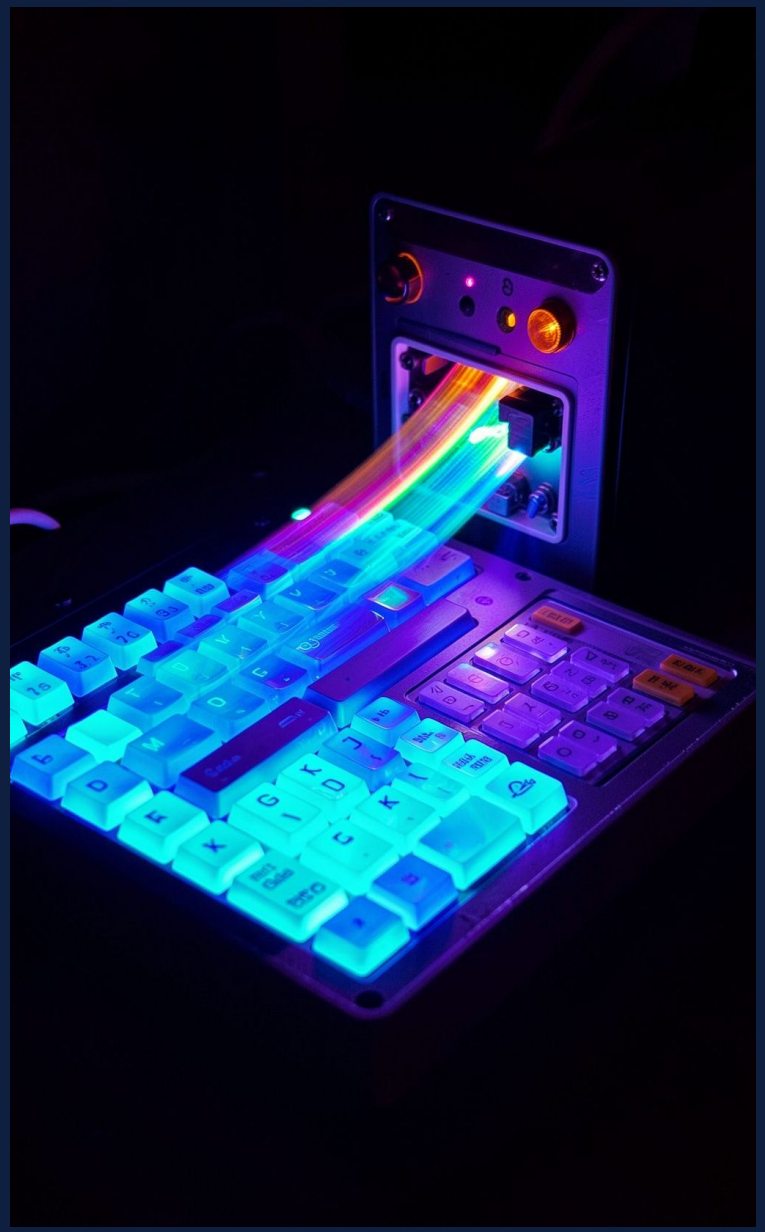
Convertir

```
int.Parse(t
exto)
```

04

Validar

Evita errores de ejecución.



Texto de entrada → valor utiliza

Made with GAMMA

06 · CONTROL DE FLUJO

Decisiones con if y switch

if / else eligen según una condición; switch / case organiza opciones discretas de forma legible. Comparadores: **==, !=, >, <, >=, <=**; condiciones compuestas: **&&** y **||**.

Condición booleana

Comparadores y operadores.

If

Rama verdadera.

Else / Default

Rama alternativa o por defecto.

Switch

Opciones discretas y casos.



Condición → camino → acción.

Made with GAMMA

Ciclos for: repetir con control

El ciclo concentra una repetición conocida en cuatro movimientos verificables.

```
for (int i = 0; i < 3; i++)
{
    Console.WriteLine(i);
}
```

Aplicar for para recorrer rangos conocidos, acumular resultados y evitar duplicar código.



Inicializar

Crear el contador.



Evaluar

Comprobar la condición.



Ejecutar

Procesar el bloque.



Actualizar

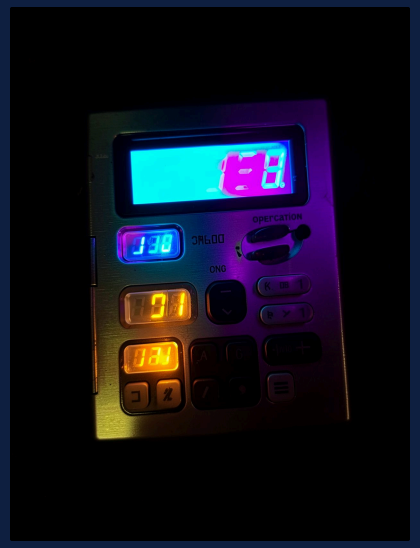
Cambiar el contador.

Made with GAMMA

08 · INTEGRACIÓN

Programa integrador: cálculo desde consola

Lee dos números, elige una operación, calcula suma, resta o multiplicación, muestra el resultado y repite el menú con un ciclo.



Un flujo completo: entrada, decisión, cálculo y repetición.

```
for (int intento = 0; intento < 3; intento++)
{
    double a = double.Parse(Console.ReadLine());
    double b = double.Parse(Console.ReadLine());
    string op = Console.ReadLine();
    double r = op switch { "+" => a + b, "-" => a - b, "*" => a * b, _ => 0 };
    Console.WriteLine($"Resultado: {r}");
}
```

Made with GAMMA

09 · CIERRE

Pensar como desarrollador.

Modelar primero. Expresar después. Controlar el flujo. Probar con datos reales.

Conclusiones: pensar como desarrollador de C#

01 Los tipos y variables modelan la información.

02 Los operadores transforman datos.

03 if y switch controlan decisiones.

04 for automatiza repeticiones.

05 La consola permite probar el flujo de extremo a extremo.

Made with GAMMA